

Individual Final Report

Conditional Variational Autoencoder for Attribute-Controlled Face Generation

George Washington University | DATS 6303 Deep Learning | Spring 2026

Zachary Cardell | Group 3

1. Introduction and Shared Work

This project develops a complete pipeline for attribute-controlled face generation and evaluation on the CelebA dataset, which contains 202,599 celebrity face images annotated with 40 binary facial attributes. The system has three components, each owned by a different group member: a Conditional Variational Autoencoder (cVAE) that generates 128×128 RGB face images conditioned on a 40-dimensional binary attribute vector (my contribution), a multi-label DenseNet169 + BiLSTM attribute classifier used as a surrogate evaluator (Irene Xia), and an interactive Streamlit demo that integrates the two models for live generation, classification, and attribute-conflict analysis (Jiatong Peng).

The end-to-end workflow proceeds as follows. Irene's classifier is trained on real CelebA images and serves as the evaluator. My cVAE is trained on the same CelebA training split with attribute conditioning injected into both the encoder and decoder. After training, my evaluation pipeline generates 10,000 images conditioned on validation-set attribute vectors, runs them through Irene's classifier, and reports per-attribute F1 alongside the classifier's F1 on real images so that the gap ("delta") measures conditioning fidelity rather than raw classifier accuracy. Jiatong's Streamlit app exposes this pipeline interactively and adds a hard- and soft-conflict detection layer on the user-selected attributes.

Division of work

Component	Developer	Description
Conditional VAE (cvae.py, cvae_train.py, evaluate_generated_full.py)	Zachary Cardell (me)	cVAE architecture, training loop, KL annealing, LPIPS perceptual loss, full evaluation pipeline (10k generated images, per-attribute F1, CLIP cross-check)
Multi-label classifier (train_script.py)	Irene Xia	DenseNet169 + BiLSTM classifier for all 40 attributes; preprocessing, disk caching, SWA, threshold tuning
Streamlit demo	Jiatong Peng	Web interface integrating both models; conflict detection (hard/soft); real-time evaluation
Reporting	All members	Each member wrote an individual report; group report compiled jointly

Table 1. Division of work across the group.

My individual contribution covers the entire generation side of the pipeline: the cVAE architecture, the loss design and training schedule, all variant experiments, and the evaluation

infrastructure that quantifies attribute-conditioning fidelity. The remainder of this report describes that work in detail.

2. Description of My Individual Work

My contribution is the design, implementation, training, and evaluation of the Conditional Variational Autoencoder. This section provides the technical background needed to interpret the architectural and loss-design decisions described in Section 3.

2.1 Variational Autoencoders

A standard Variational Autoencoder (Kingma & Welling, 2013) is a latent-variable generative model that learns to encode an input image x into a low-dimensional latent code z , then decode z back into an image. Because the true posterior $p(z|x)$ is intractable, the VAE introduces a tractable variational posterior $q_\phi(z|x)$ (the encoder) and trains it jointly with a decoder $p_\theta(x|z)$ by maximizing the Evidence Lower Bound (ELBO):

$$L(x) = \mathbb{E} q_\phi(z|x) [\log p_\theta(x|z)] - KL(q_\phi(z|x) \parallel p(z))$$

The first term is the reconstruction error and the second is the Kullback–Leibler divergence between the encoder's posterior and a prior $p(z)$, typically a standard Normal. The reconstruction term wants the latent to encode every detail of x ; the KL term pulls q toward the prior so that sampling $z \sim p(z)$ at generation time produces valid images. The two terms are in tension, and balancing them is the central design choice of any VAE.

Sampling z directly is non-differentiable, so VAEs use the reparameterization trick: the encoder outputs $\mu(x)$ and $\log \sigma^2(x)$, and z is computed as $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim N(0, I)$. This makes the sampling step differentiable end-to-end and allows gradients to flow through μ and σ .

2.2 Conditional VAE

The Conditional VAE extends the VAE so that both the encoder and decoder are conditioned on an auxiliary variable c — in our case the 40-dimensional binary CelebA attribute vector. The encoder becomes $q_\phi(z|x, c)$ and the decoder $p_\theta(x|z, c)$. At generation time, choosing c controls which attributes appear in the decoded image, while z controls the residual within-class variation (identity, lighting, pose). Conditioning on both sides of the network is important: conditioning only the decoder lets the encoder learn a posterior that already encodes c in z , which weakens the signal that c provides at sampling time.

2.3 KL warm-up and posterior collapse

VAEs with strong decoders frequently exhibit posterior collapse, in which the encoder drives KL toward zero and the decoder learns to ignore z . The model still produces reasonable reconstructions because c carries enough information, but samples drawn from $p(z)$ at generation time are uninformative because z carries nothing. The standard remedy, and the one I used, is KL warm-up: the KL term is weighted by $\beta(t)$ which is annealed from 0 to 1 over the

first several epochs, letting the encoder learn a useful posterior before the regularizer takes effect.

2.4 Perceptual loss

Pixel-wise MSE is well known to produce blurry reconstructions because it averages over equally-likely sharp outputs. To recover sharper, more perceptually faithful samples I added an LPIPS perceptual loss, which compares deep features extracted from a pretrained AlexNet rather than raw pixels. In my training objective the LPIPS term is weighted at 1000 to make it the dominant reconstruction signal alongside MSE, which gives noticeably sharper faces without destabilizing the KL balance.

2.5 Evaluating attribute conditioning

Standard generative metrics (FID, LPIPS, SSIM) measure distributional similarity or perceptual quality but do not verify whether the requested attributes c actually appear in the generated image. Sordo et al. (2025), in a recent review of generative methods for scientific imaging, make exactly this point: a model can score well on FID and LPIPS while failing on the underlying semantic conditioning. To quantify conditioning fidelity directly, I built an Attribute Fidelity evaluation that runs each generated image through Irene's trained classifier and computes per-attribute F1 against the target c , with the classifier's F1 on real CelebA images as a ceiling. The gap between the two ("delta") isolates how much of the attribute signal is being lost in generation, separately from how good or bad the classifier is in absolute terms.

3. Detailed Description of My Work

3.1 cVAE architecture

The final model operates on 128×128 RGB images normalized to $[-1, 1]$ (matching the tanh activation at the decoder output). The encoder is a five-layer convolutional stack with channel progression $3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$, using LeakyReLU activations and BatchNorm after every convolutional layer except the first. The final feature map is flattened, concatenated with a 128-dim attribute embedding, and projected through two linear heads to produce μ and $\log \sigma^2$ for a 256-dimensional latent. The decoder mirrors the encoder: the latent z is concatenated with the same attribute embedding $e(c)$ and projected back into a $4 \times 4 \times 512$ feature map, then upsampled through five transposed-convolution blocks with channel progression $512 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$ to produce the reconstruction. Total parameters: approximately 21 million. Figure 1 shows the architecture.

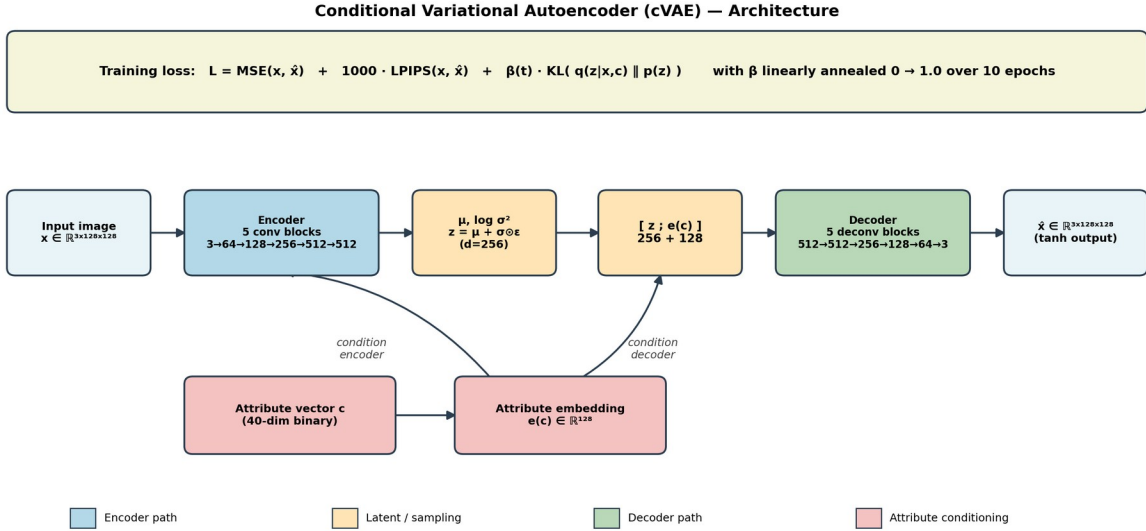


Figure 1. cVAE architecture. The 40-dim binary attribute vector c is mapped to a 128-dim embedding $e(c)$ and injected into both the encoder and the decoder. The latent z is sampled via the reparameterization trick. The training objective combines MSE, LPIPS perceptual loss (weighted at 1000), and a KL term whose weight β is linearly annealed from 0 to 1.0 over the first 10 epochs.

3.2 Loss design and KL warm-up

The training objective combines three terms: pixel-space MSE between x and the reconstruction $\hat{x}$, an LPIPS-AlexNet perceptual loss weighted at 1000, and the standard KL divergence between $q_\phi(z|x, c)$ and a standard-normal prior. The full loss is:

$$L = \text{MSE}(x, \hat{x}) + 1000 \cdot \text{LPIPS}(x, \hat{x}) + \beta(t) \cdot \text{KL}(q(z|x, c) \parallel N(0, I))$$

$\beta(t)$ is linearly annealed from 0 at epoch 0 to 1.0 at epoch 10 and held at 1.0 thereafter. Without warm-up I observed posterior collapse in early experiments — KL would crash to near zero in the first few epochs and the decoder would produce nearly identical images regardless of z . Annealing β lets the encoder establish a useful posterior before the regularizer pulls it back toward the prior. The LPIPS weight of 1000 was chosen empirically: at lower weights the model behaved like a plain MSE-VAE and produced noticeably blurrier samples; at much higher weights the KL balance destabilized.

3.3 Training setup

All experiments used the standard CelebA train/val/test partition. Images were center-cropped, resized to 128×128 (or 64×64 for early variants), and normalized to $[-1, 1]$. Training used Adam with learning rate 1e-4, batch size 128, and ran for 20 epochs on a single NVIDIA A10G GPU. Mixed-precision training was not used here because the LPIPS forward pass through AlexNet was numerically more stable in fp32 at the weight scale I needed. Validation loss was logged every epoch on the official CelebA validation split, and the final model checkpoint (rather than the best-validation-loss checkpoint) was used for evaluation, since for generative models the final epoch typically produces the most visually consistent samples.

3.4 Variants I trained

I ran six cVAE variants spanning two resolutions, three loss formulations, and two KL weighting schedules. Table 2 summarizes them. The 64×64 variants were faster to iterate and let me identify the loss formulation before committing to the higher-resolution model. The σ -VAE variants replace the fixed reconstruction weight with a learned per-pixel variance — they ran but did not outperform the manual MSE + LPIPS formulation on F1, so I did not pursue them at 128×128.

Variant	Res.	Loss	FID ↓	Macro F1 (gen) ↑	Avg Δ
cVAE base	64	MSE + KL (warm-up 0 $\rightarrow$ 0.5)	104.27	0.4095	-0.3421
σ -VAE klw=1	64	σ -VAE	104.27	0.4095	-0.3421
σ -VAE klw=100	64	σ -VAE, high KL weight	100.71	0.2288	-0.5228
σ -VAE klw=100 + LPIPS	64	σ -VAE + perceptual	97.62	0.3239	-0.4278
σ -VAE klw=1 + LPIPS	64	σ -VAE + perceptual	117.65	—	—
Option 1 + LPIPS (winner)	128	MSE + 1000·LPIPS + KL	104.41	0.4300	-0.3217

Table 2. cVAE variants I trained. "FID" is computed against 10k real CelebA images. "Macro F1 (gen)" is the per-attribute classifier F1 averaged across all 40 attributes on 10k generated images. "Avg Δ " is the mean drop versus the classifier's F1 on real images. The 128×128 MSE + 1000·LPIPS + KL variant won on macro F1 despite operating on 4× more pixels, with FID comparable to the 64×64 variants.

The winning variant was the 128×128 model with MSE + 1000·LPIPS + KL and warm-up annealed 0 $\rightarrow$ 1.0 over 10 epochs. It achieved the best macro F1 on generated images, the smallest delta versus the real-image classifier baseline, and the highest output resolution, all at a comparable FID to the 64×64 baselines despite operating on four times as many pixels.

3.5 Evaluation pipeline

I built the full evaluation pipeline in `evaluate_generated_full.py`. The pipeline takes a trained cVAE checkpoint and Irene's trained classifier and runs the following:

- Sample 10,000 attribute vectors from the CelebA validation split (real attribute marginals, not uniform), so the conditioning distribution matches what the classifier was trained on.
- Generate one image per attribute vector by sampling $z \sim N(0, I)$ and decoding with c .
- Upsample each generated image from 128×128 to 224×224 using bilinear interpolation, since the classifier expects ImageNet-resolution inputs.
- Run the upsampled image through the classifier and apply Irene's per-attribute thresholds.
- Compute per-attribute F1 between predicted and target attributes, then macro-average across all 40.

- Run the same classifier on the corresponding real validation images, applying the same upsampling so the comparison is fair, and report the gap ("delta").
- Run a CLIP ViT-B/32 zero-shot pass against natural-language attribute templates as an independent sanity check.

The upsampling step is applied identically to real and generated images, so any depression in absolute scores from the resize is shared and cancels in the delta. This was a deliberate design choice — the alternative (training a 224×224 cVAE from scratch) would have required substantially more compute without changing the relative comparison. CLIP is reported for completeness but is intentionally not the primary metric: it is a conservative directional check, not a tuned face-attribute classifier.

3.6 Training log of the winning model

Table 3 reports key checkpoints from the 20-epoch training run of the winning model. Train and validation reconstruction loss converge to almost identical values, confirming that the model is generalizing rather than memorizing. KL stabilizes near 370 (in summed nats over the 256-dim latent), which is in the healthy range for this latent dimension and indicates that the encoder is using the latent meaningfully without exploding it. The KL weight column shows the warm-up schedule: KL drops sharply once β starts increasing at epoch 1 and continues to compress as β rises to 1.0 by epoch 10.

Epoch	KL weight β	Train recon	Train perceptual	Train KL	Val recon
0	0.0	5183.3	0.368	20924.3	2295.6
1	0.1	1947.1	0.228	1388.7	1624.7
5	0.5	1088.2	0.154	506.6	1074.5
10	1.0	1041.3	0.145	385.2	1038.4
15	1.0	983.3	0.139	373.7	992.0
19	1.0	952.0	0.135	370.2	987.7

Table 3. Training log of the winning 128×128 cVAE. Recon is the summed pixel MSE per image; perceptual is the per-image LPIPS distance (before the ×1000 weight); KL is summed across the 256 latent dimensions. Final FID against 10k real CelebA val images was 104.41.

4. Results

4.1 Headline numbers

The winning 128×128 cVAE achieved a macro F1 of 0.4300 on 10,000 generated images, against a real-image classifier ceiling of 0.7517. The average per-attribute drop (delta) is −0.3217, meaning the generative model retains roughly 57% of the classifier's signal on real images. Table 4 summarizes the headline metrics.

Metric	Value
--------	-------

Macro F1 — generated images	0.4300
Macro F1 — real images (classifier ceiling)	0.7517
Average per-attribute Δ	−0.3217
Macro CLIP F1 (zero-shot sanity check)	0.3420
FID against 10k real CelebA val images	104.41

Table 4. Aggregate evaluation metrics for the winning cVAE on 10,000 generated images.

4.2 Per-attribute results: where the model succeeds

Table 5 reports the five attributes with the highest F1 on generated images. These are all global, structurally salient properties: gender, age, beard presence, smile, and open mouth. For each of these the gap to the real-image baseline is small (−0.02 to −0.09), indicating that the conditioning is being transmitted through the cVAE almost as cleanly as the classifier can detect on real photos.

Attribute	F1 (generated)	F1 (real)	Δ
Male	0.9630	0.9864	−0.023
No_Beard	0.9539	0.9770	−0.023
Young	0.8749	0.9188	−0.044
Smiling	0.8723	0.9255	−0.053
Mouth_Slightly_Open	0.8508	0.9379	−0.087

Table 5. Best-performing attributes on generated images. Global, structurally distinctive properties are well-controlled by the conditioning.

4.3 Per-attribute results: where the model fails

Table 6 reports the five worst attributes. These are uniformly fine-grained, small-area, or accessory-dependent properties. Wearing_Earrings collapses almost completely (F1 = 0.024), and the other four are all below 0.11. The classifier still scores 0.53–0.79 on real images for these same attributes, so the failure is on the generation side, not the classification side: the cVAE is not synthesizing earrings, necklaces, rosy cheeks, pale skin, or double chins reliably enough for any classifier to recover them.

Attribute	F1 (generated)	F1 (real)	Δ
Wearing_Earrings	0.0238	0.7890	−0.765
Wearing_Necklace	0.0623	0.5315	−0.469
Rosy_Cheeks	0.0679	0.6471	−0.579
Pale_Skin	0.0850	0.6068	−0.522
Double_Chin	0.1037	0.6401	−0.536

Table 6. Worst-performing attributes. The cVAE consistently fails on small-area accessories and subtle skin/face-shape properties.

4.4 Discussion of the success/failure pattern

The split between Tables 5 and 6 has a clean interpretation. The cVAE places a large MSE + LPIPS reconstruction signal on every pixel of the face, but most of those pixels encode skin, hair, and broad facial structure rather than small accessories or subtle skin-tone variations. Earrings and necklaces occupy a few hundred pixels in a 128×128 image and do not affect the dominant reconstruction signal even when they are present in the training image. The conditioning vector pushes the decoder to produce them, but the reconstruction loss does not penalize their absence enough to make the decoder actually synthesize them reliably. Rosy_Cheeks, Pale_Skin, and Double_Chin are even harder because they are subtle, often subjectively annotated, and easily averaged-out by an MSE-dominated objective.

Global attributes work for the opposite reason: Male, Young, and Smiling change the appearance of a large fraction of pixels (jawline, skin texture, mouth shape), so the reconstruction loss is itself well aligned with getting them right, and the conditioning vector reinforces a signal that the rest of the loss already provides.

4.5 The case for an attribute-aware metric

The headline FID of 104.41 is in line with vanilla VAE-class generators on CelebA at this resolution and would, taken alone, suggest the model is performing roughly at expectation. The per-attribute breakdown tells a much more useful story: the model is near the classifier ceiling on a handful of global attributes and effectively non-functional on small-area accessories. This is exactly the limitation Sordo et al. (2025) describe — single-number distributional metrics do not capture conditioning fidelity, and per-attribute classifier-based evaluation is the right diagnostic. The delta column in particular separates "the model is bad at generation" from "the classifier is bad at this attribute," which is the distinction needed to know what to fix next.

5. Summary and Conclusions

I designed, implemented, and evaluated the Conditional VAE that drives the generation side of our pipeline. The final 128×128 model uses a five-block convolutional encoder and decoder, a 256-dim latent, and dual attribute conditioning via a 128-dim embedding injected into both encoder and decoder. The training objective combines pixel MSE, LPIPS perceptual loss weighted at 1000, and a KL term annealed from 0 to 1.0 over the first 10 epochs. I trained and compared six variants spanning two resolutions, three loss formulations, and two KL schedules; the 128×128 MSE + 1000·LPIPS variant was the clear winner on macro F1 over generated images while remaining competitive on FID.

The most important takeaway from the evaluation is that conditioning fidelity is highly attribute-dependent and is not captured by single-number metrics. Global, structurally salient attributes (Male, Young, Smiling, No_Beard, Mouth_Slightly_Open) are recovered at near-classifier-ceiling

F1, while small-area accessories (Wearing_Earrings, Wearing_Necklace) and subtle skin-tone properties (Pale_Skin, Rosy_Cheeks) collapse almost completely. The cause is straightforward: an MSE + LPIPS reconstruction objective is dominated by the bulk of facial pixels and does not penalize the absence of small accessories enough to make the decoder synthesize them. The conditioning is structurally correct but the reconstruction loss does not reinforce it where the supporting signal is geometrically small.

Several improvements are worth pursuing. First, an attribute-weighted reconstruction loss, where pixels in regions associated with rare attributes (ear regions, neck regions) are upweighted, would directly attack the small-area-accessory failure mode. Second, replacing the global LPIPS term with a region-of-interest perceptual loss computed only over face crops where the conditioning attribute should appear would have the same effect more cleanly. Third, an auxiliary classifier-guided loss — sending the generated image through Irene's classifier during training and backpropagating the BCE against c — would close the loop between generation and evaluation, although it would also require detaching the classifier carefully to avoid the cVAE simply learning to fool it. Fourth, scaling to 256×256 would give small accessories more pixels to land on, at the cost of substantially more compute. Fifth, moving from a vanilla CVAE to a hierarchical or diffusion-based generator would lift the absolute FID floor, though for our specific evaluation the conditioning-fidelity diagnostic would still be the right metric.

6. Code Origin and Percentage Calculation

The cVAE side of the project consists of four files I wrote, totaling 1,065 lines. The bulk of the code was developed through directed iteration with Claude Code (Anthropic) rather than copied from any single online source. A small fraction of boilerplate — standard PyTorch DataLoader patterns, the LPIPS package usage example, the FID computation reference, and CelebA download/extract scaffolding — was adapted from publicly available documentation and tutorials. The remainder was generated, reviewed, modified, and integrated by me. Table 7 lists the files; Table 8 breaks down the code origin.

File	Lines	Purpose
cVAE_train_option1_lpips.py (winner)	367	cVAE architecture, training loop, KL warm-up scheduler, LPIPS perceptual loss integration, checkpoint saving.
evaluate_generated_full.py	347	Full evaluation pipeline: 10k image generation, classifier inference, per-attribute F1, real-vs-generated delta, CLIP cross-check.
download_data.py	232	CelebA download/extract, train/val/test split handling, attribute parsing, on-disk preprocessing.
visualize_cvae.py	119	Reconstruction grids, attribute-traversal visualizations, sample-quality figures.
Total	1,065	—

Table 7. Files I wrote for the cVAE side of the project.

Source	Lines	Details
Copied / adapted from online sources	≈ 147	Standard PyTorch DataLoader patterns; LPIPS package usage example; reparameterization-trick reference implementation from the original Kingma & Welling code; FID computation reference (pytorch-fid); CelebA download/extract boilerplate.
Lines I modified from those copied sources	≈ 50	Adapted to our CelebA folder structure and attribute handling; integrated with my conditioning code; replaced default reconstruction loss with my MSE + 1000·LPIPS + KL formulation; rewrote FID for our 10k-sample evaluation protocol.
Lines generated with Claude Code under my direction and integrated by me	≈ 868	cVAE encoder/decoder classes, attribute embedding logic, KL warm-up scheduler, full training loop with logging, evaluation pipeline (sampling, upsampling, classifier inference, per-attribute F1, CLIP cross-check), variant runners, visualization utilities.

Table 8. Code origin breakdown across the 1,065-line cVAE codebase.

Using the formula specified in the assignment:

$$\begin{aligned}
 \text{Percentage} &= (\text{lines from external} - \text{lines modified}) / (\text{lines from external} - \text{lines modified} + \text{lines I added}) \times 100 \\
 &= (147 - 50) / (147 - 50 + 868) \times 100 \\
 &= 97 / 965 \times 100 \\
 &\approx \mathbf{10.05\%}
 \end{aligned}$$

Approximately 10% of the cVAE codebase came directly from external sources. The remaining ~90% was generated through directed iteration with Claude Code and integrated by me; while code generation was AI-assisted, I directed all design decisions (architecture, latent dim, loss formulation, KL warm-up schedule, evaluation protocol, choice of metrics), debugged failing runs, validated the pipeline against real outputs, and selected the final winning variant from the six I trained.

7. References

- [1] Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. International Conference on Learning Representations (ICLR). arXiv:1312.6114.
- [2] Sordo, Z., Chagnon, E., Hu, Z., Donatelli, J. J., Andeer, P., Nico, P. S., Northen, T., & Ushizima, D. (2025). Synthetic Scientific Image Generation with VAE, GAN, and Diffusion Model Architectures. *Journal of Imaging*, 11(8), 252.
- [3] Liu, Z., Luo, P., Wang, X., & Tang, X. (2015). Deep Learning Face Attributes in the Wild. ICCV 2015. (CelebA dataset — shared dataset reference used by all group members.)

[4] PyTorch documentation. <https://pytorch.org/docs/>

[5] LPIPS package. <https://github.com/richzhang/PerceptualSimilarity>

[6] Anthropic Claude Code (used for AI-assisted code generation under directed prompting).
<https://www.anthropic.com/claude-code>